IU Digital de Antioquia

Actividad de Aprendizaje 1 : Fases 1 y 2 de la metodología CRISP-DM

Curso : Programación para análisis de datos

Estudiante:

Juan Fernando Cataño Higuita

Óscar Javier García García

Rober Andrés Castillo Gaviria

Juan David Ramírez García

Docente:

Ana María López Moreno

Medellín

IU digital de Antioquia

3 de mayo de 2026

ÍNDICE

1. Definición de la necesidad.....4
2. Diseño de la necesidad.....4
3. Modelos de diseño..... 5
4. Conexión y carga de pruebas de la base de datos..... 6
5. Repositorio para ver el script..... 11
Link video presentación.....11
Conclusiones..... 11

1. Definición de la necesidad

Históricamente, el homicidio en Colombia se ha analizado desde la perspectiva del conflicto armado y el narcotráfico. Sin embargo, esta violencia no es uniforme; también se manifiesta mediante diversas dinámicas en contextos rurales y urbanos a través de hurtos, riñas, sicariatos, etc.

El objetivo de este trabajo es analizar cómo este otro tipo de homicidios ha afectado a Colombia a través del tiempo y sus regiones, utilizando el dataset para entender la problemática desde su modalidad y causas.

Pregunta de negocio o de la problemática: **¿qué patrones de homicidio existen fuera del conflicto?**

2. Diseño de la necesidad

Objetivos de negocio:

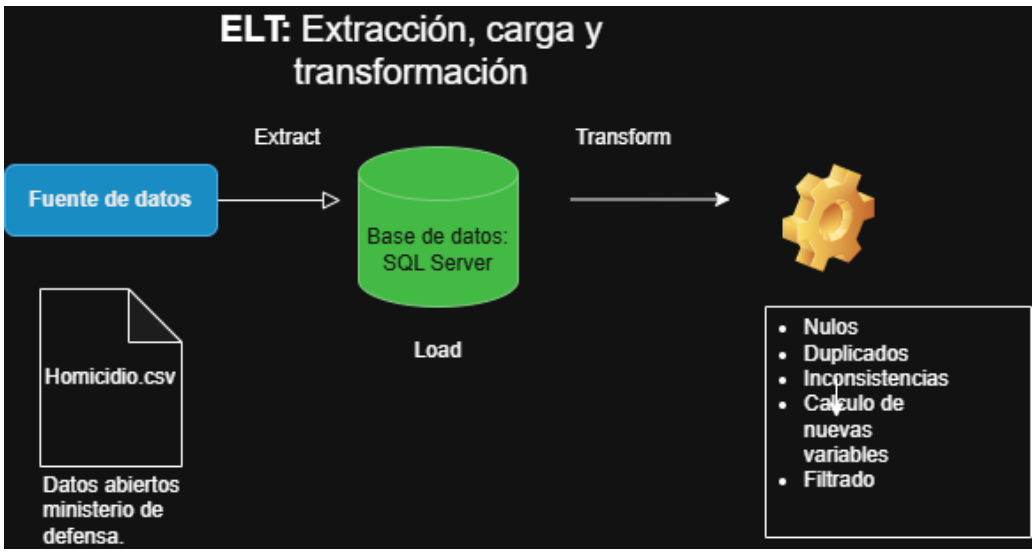
- Identificar los tipos de violencias por las diferentes regiones de Colombia para apoyar la toma de decisiones en seguridad ciudadana.
- Clasificar los homicidios por modalidad y zona para distinguir la violencia común de la violencia por conflicto armado.

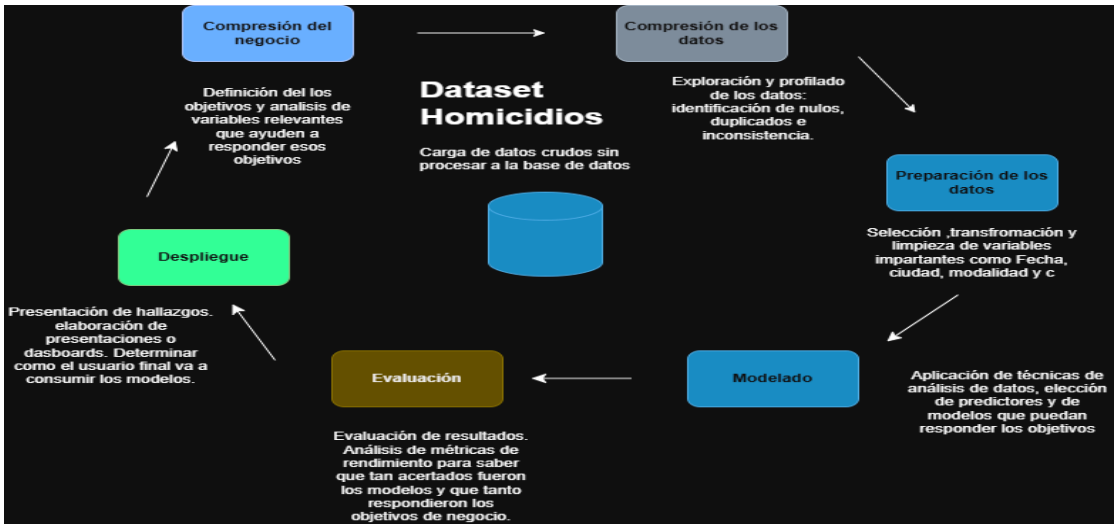
Fuentes de datos relevantes:

dataset:https://www.datos.gov.co/Seguridad-y-Defensa/HOMICIDIO/m8fd-ahd9/about_data

Fuente: Datos abiertos del Ministerio de Defensa. Última actualización 24 de abril del 2026

https://www.mindefensa.gov.co/





3. Modelos de diseño

Se va a utilizar un modelo estrella para responder a los objetivos de negocio, mediante las siguientes tablas: **Hechos_Homicidios, Dim_Modalidad, Dim_Municipio, Dim_Departamento y Dim_arma**

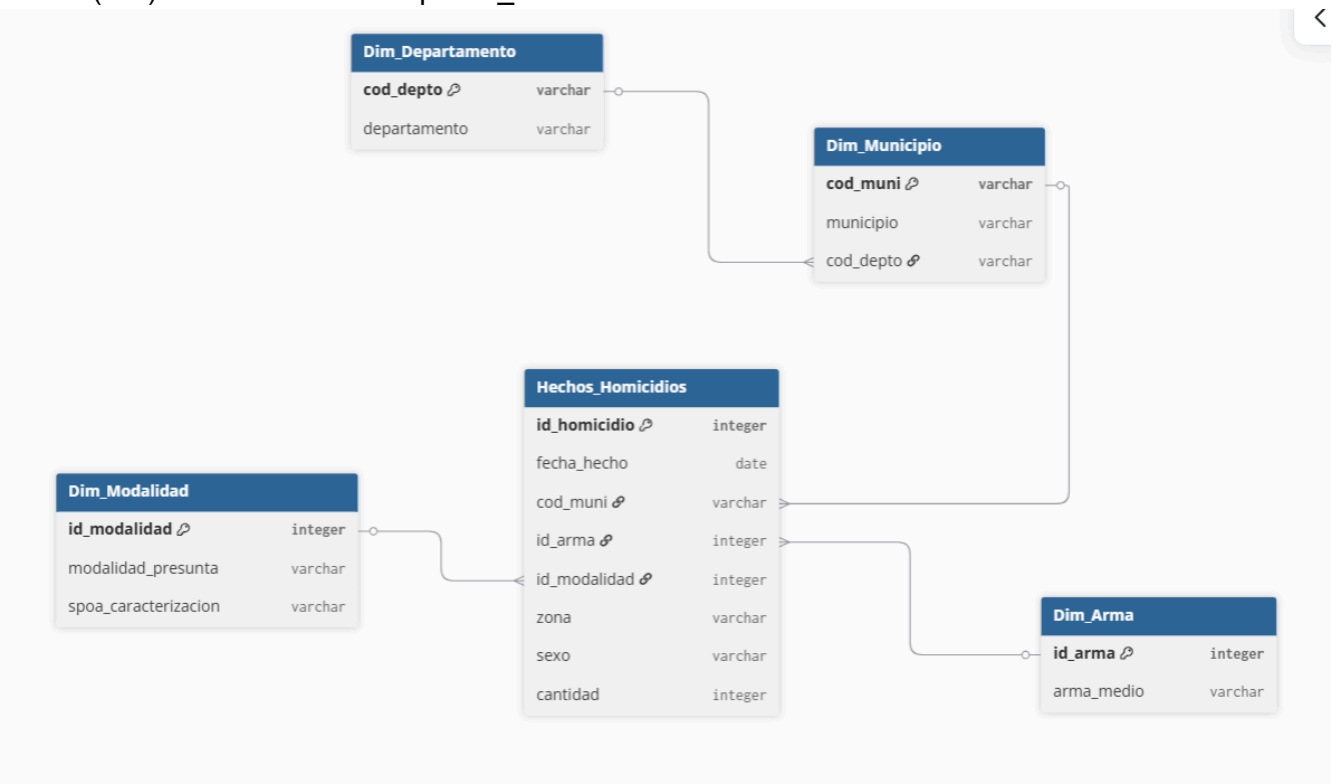
Relaciones del modelo

- entre la tabla **Hechos_Homicidios** y **Dim_Modalidad** se establació una relación de Muchos a Uno (N:1) a través de los campos **id_modalidad** como Clave Foránea en Hechos_Homicidios y Clave Primaria en la tabla **Dim_Modalidad.**

- Entre la tabla **Hechos_Homicidios** y **Dim_Municipio** se estableció una relación de Muchos a Uno (N:1) a través de los campos cod_muni como clave foránea y clave primaria en la tabla **Dim_Municipio.**

- Entre la tabla Dim_Municipio y Dim_Departamento existe una relación de Muchos a Uno (N:1) a través de los campos cod_depto.

- entre la tabla **Hechos_Homicidios** y **Dim_Arma** se establece una relación de Muchos a Uno (N:1) a través de los campos id_arma.



4. Conexión y carga de pruebas de la base de datos

Vamos a cargar nuestro dataset desde el archivo *homicidio.csv* a una base de datos llamada *HomicidiosDB* en SQL Server, donde cargaremos los datos crudos en unas tablas **Staging** para una futura transformación y limpieza.

Script para cargar datos a la base de datos

1. Importamos librería y configuración de conexión a la base de datos

```python
import pandas as pd
from sqlalchemy import create_engine, text

# Configuración de conexión
file_path = 'homicidio.csv'
server = r'LAPTOP-5G1JGRV9\SQLEXPRESS'
database = 'HomicidiosDB'

engine = create_engine(f'mssql+pyodbc://{server}/{database}?driver=ODBC+Driver+17+for+SQL+Server')
```

2. Creación de tablas en la base de datos y cargamos datos desde el csv

```python
def load_and_upload():

    print("Cargando datos desde CSV...")
    # Cargamos y limpiamos espacios en los nombres de las columnas
    df = pd.read_csv(file_path)
    df.columns = df.columns.str.strip()

    create_tables_query = """
    DROP TABLE IF EXISTS Staging_Homicidios;
    DROP TABLE IF EXISTS Staging_Municipio;
    DROP TABLE IF EXISTS Staging_Departamento;
    DROP TABLE IF EXISTS Staging_Arma;
    DROP TABLE IF EXISTS Staging_Modalidad;

    CREATE TABLE Staging_Departamento (
        cod_depto VARCHAR(50) PRIMARY KEY,
        departamento VARCHAR(255)
    );

    CREATE TABLE Staging_Municipio (
        cod_muni VARCHAR(50) PRIMARY KEY,
        municipio VARCHAR(255),
        cod_depto VARCHAR(50),
        FOREIGN KEY (cod_depto) REFERENCES Staging_Departamento(cod_depto)
    );

    CREATE TABLE Staging_Arma (
        id_arma INT PRIMARY KEY IDENTITY(1,1),
        arma_medio VARCHAR(255)
    );

    CREATE TABLE Staging_Homicidios (
        id_homicidio INT PRIMARY KEY IDENTITY(1,1),
        fecha_hecho DATE,
        cod_muni VARCHAR(50),
        id_arma INT,
        id_modalidad INT,
        zona VARCHAR(50),
        sexo VARCHAR(50),
        cantidad INT,
        FOREIGN KEY (cod_muni) REFERENCES Staging_Municipio(cod_muni),
        FOREIGN KEY (id_arma) REFERENCES Staging_Arma(id_arma),
        FOREIGN KEY (id_modalidad) REFERENCES Staging_Modalidad(id_modalidad)
    );
    """

    with engine.connect() as connection:
        print("Limpiando y creando tablas...")
        connection.execute(text(create_tables_query))
        connection.commit()
```

3. Creación de las dimensiones desde el dataframe que después vamos a cargar en las tablas de staging.

```
# --- CREACIÓN DE DIMENSIONES ---

# Departamento: BORRAMOS DUPLICADOS PARA ASEGURAR QUE COD_DEPTO SEA ÚNICO
depto_df = df[['COD_DEPTO', 'DEPARTAMENTO']].drop_duplicates(subset=['COD_DEPTO'])
depto_df.columns = ['cod_depto', 'departamento']

# MUNICIPIO: BORRAMOS DUPLICADOS PARA ASEGURAR QUE COD_MUNI SEA ÚNICO
muni_df = df[['COD_MUNI', 'MUNICIPIO', 'COD_DEPTO']].drop_duplicates(subset=['COD_MUNI'])
muni_df.columns = ['cod_muni', 'municipio', 'cod_depto']

# Arma
arma_df = df[['ARMA MEDIO']].drop_duplicates()
arma_df.columns = ['arma_medio']

# Modalidad: Limpiamos espacios para evitar problemas de mapeo
mod_df = df[['MODALIDAD PRESUNTA']].drop_duplicates()
mod_df.columns = ['modalidad_presunta']
```

4. Cargando los datos a staging

```
#---Carga de datos a staging---

print("Cargando dimensiones de staging a SQL...")

depto_df.to_sql('Staging_Departamento', engine, if_exists='append', index=False)
muni_df.to_sql('Staging_Municipio', engine, if_exists='append', index=False)
arma_df.to_sql('Staging_Arma', engine, if_exists='append', index=False)
mod_df.to_sql('Staging_Modalidad', engine, if_exists='append', index=False)
```

5. Cargando datos a staging hechos.

```
# --- CARGA DE HECHOS ---
print("Mapeando IDs y cargando Hechos...")

db_armas = pd.read_sql("SELECT id_arma, arma_medio FROM Staging_Arma", engine)
db_mods = pd.read_sql("SELECT id_modalidad, modalidad_presunta FROM Staging_Modalidad", engine)

hechos_df = df.copy()
hechos_df = hechos_df.merge(db_armas, left_on='ARMA MEDIO', right_on='arma_medio')
hechos_df = hechos_df.merge(db_mods, left_on='MODALIDAD PRESUNTA', right_on='modalidad_presunta')

hechos_final = hechos_df[['FECHA HECHO', 'COD_MUNI', 'id_arma', 'id_modalidad', 'ZONA', 'SEXO', 'CANTIDAD']]
hechos_final.columns = ['fecha_hecho', 'cod_muni', 'id_arma', 'id_modalidad', 'zona', 'sexo', 'cantidad']
hechos_final['fecha_hecho'] = pd.to_datetime(hechos_final['fecha_hecho'], dayfirst=True)

hechos_final.to_sql('Staging_Homicidios', engine, if_exists='append', index=False)
print("¡Proceso completado sin errores!")

if __name__ == "__main__":
    load_and_upload()
```

Prueba de que los datos se cargaron correctamente a la base de datos.

Tabla **Staging_Homicidios** que contiene los datos crudos que van a ser después cargados a **Hechos_Homicidios**

```sql
select top 5 * from Staging_Homicidios;
```

154 %

Results | Messages

	id_homicidio	fecha_hecho	cod_muni	id_arma	id_modalidad	zona	sexo	cantidad
1	1	2003-01-01	91001	1	1	RURAL	MASCULINO	1
2	2	2003-01-01	5034	2	1	RURAL	MASCULINO	1
3	3	2003-01-01	5045	1	2	URBANA	MASCULINO	1
4	4	2003-01-01	5088	2	1	URBANA	MASCULINO	1
5	5	2003-01-01	5088	2	1	URBANA	MASCULINO	1

De igual forma para las tablas Staging_Arma, Staging_Departamento, Staging_Modalidad y Staging_Municipio.

```sql
select top 5 * from Staging_Arma;
```

154 %

Results | Messages

	id_arma	arma_medio
1	1	ARMA BLANCA / CORTOPUNZANTE
2	2	ARMA DE FUEGO
3	3	CONTUNDENTES
4	4	ARTEFACTO EXPLOSIVO/CARGA DINAMITA
5	5	NO REPORTADO

```sql
select top 5 * from Staging_Departamento;
```

154 %

Results | Messages

	cod_depto	departamento
1	11	BOGOTA D.C.
2	13	BOLIVAR
3	15	BOYACA
4	17	CALDAS
5	18	CAQUETA

```sql
select top 5 * from Staging_Modalidad;
```

154 %

Results | Messages

	id_modalidad	modalidad_presunta
1	1	NO REPORTADA
2	2	RIÑAS
3	3	RIÑA ENTRE ESPOSOS
4	4	SICARIATO
5	5	ATACADOS POR SUBVERSION

```sql
select top 5 * from Staging_Municipio;
```

154 %

Results | Messages

	id_modalidad	modalidad_presunta
1	1	NO REPORTADA
2	2	RIÑAS
3	3	RIÑA ENTRE ESPOSOS
4	4	SICARIATO
5	5	ATACADOS POR SUBVERSION

5. Repositorio para ver el script

https://github.com/JuanDavidRamirezGarcia/Programaci-n-para-el-an-lisis-de-datos-2026-1

Link video presentación

https://youtu.be/p03P5hn3TNM

Conclusiones

- La aplicación de las primeras dos fases de la metodología CRISP-DM fue fundamental para delimitar el alcance y comprender la problemática de los homicidios en Colombia. Esto nos permitió establecer una perspectiva de análisis cuantitativa, enfocada en entender el impacto de este fenómeno según las diferentes regiones del país.
- Se realizó el modelo de diseño para gestionar los datos de manera organizada a través de la creación de un diagrama de ETL y entidad-relación.
- Se ejecutó sin errores el script para la creación del modelo estrella y carga de datos en la base de datos.